

# Debezium CDC Pipeline: Red Hat OpenShift Architecture & Operations Guide

## Table of Contents

- 1. [Executive Summary](#)
- 2. [Architecture Overview](#)
- 3. [System Components](#)
- 4. [Deployment Architecture](#)
- 5. [Configuration Reference](#)
- 6. [Data Flow & Transformations](#)
- 7. [Known Issues & Resolutions](#)
- 8. [Operational Procedures](#)

## Executive Summary

This document describes a Change Data Capture (CDC) pipeline that replicates data from AWS Managed Streaming for Kafka (MSK) to enterprise databases (Db2 for z/OS and MSSQL) using Red Hat Debezium on OpenShift. The implementation addresses critical constraints imposed by AWS MSK's read-only access policy and database-specific identifier naming rules.

### Key Achievements

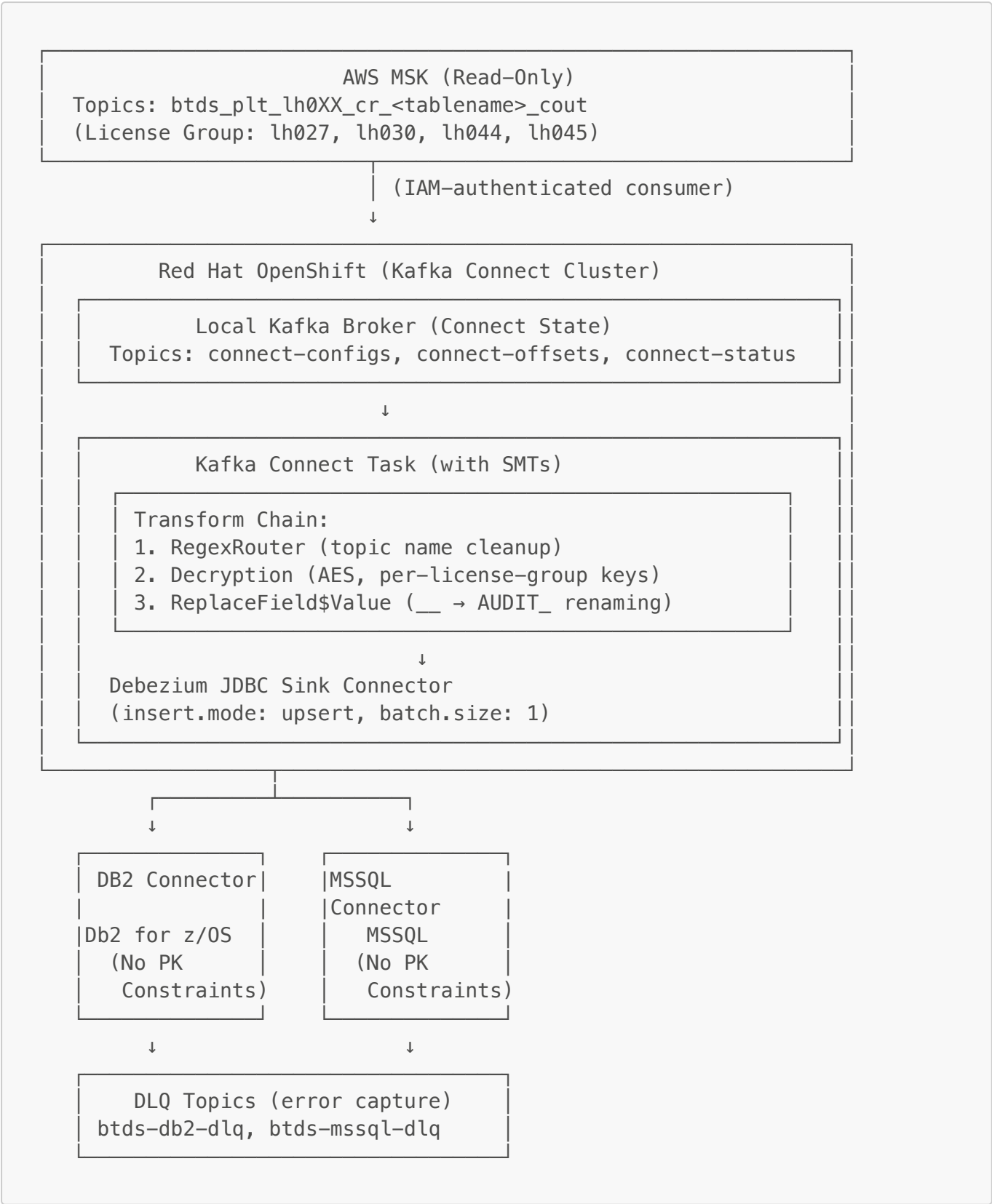
- **Decoupled architecture:** AWS MSK (read-only source) → Local Kafka (state management) → Sink databases
- **Multi-tenant support:** Handles 4 license groups (lh027, lh030, lh044, lh045) with isolated connectors
- **Encryption-aware:** Custom SMT for per-topic decryption with AES per-license-group key resolution
- **Database compatibility:** Solves Db2 z/OS identifier restrictions via field renaming
- **Reliable delivery:** Batch processing, error handling, and dead-letter queue (DLQ) infrastructure

### Pain Points Solved

| Issue                            | Root Cause                                   | Solution                                             |
|----------------------------------|----------------------------------------------|------------------------------------------------------|
| Connector crashes on Db2 inserts | ___ prefixed CDC columns invalid in Db2 z/OS | ReplaceField\$Value SMT to rename to <b>AUDIT_*</b>  |
| AWS MSK read-only constraint     | MSK internal topics not writable             | Local Kafka cluster for Connect state                |
| Decryption SMT fragmentation     | Multiple SMTs per topic                      | Unified <b>Decryption</b> SMT with topic→key mapping |

# Architecture Overview

## Logical Flow Diagram



## Design Rationale

### Why a Local Kafka Cluster?

AWS MSK is configured with **read-only** access, which restricts write operations to internal topics. Kafka Connect requires persistent state management via internal topics:

| Internal Topic               | Purpose                             | Why Read-Only MSK Won't Work                    |
|------------------------------|-------------------------------------|-------------------------------------------------|
| <code>connect-configs</code> | Connector configuration persistence | Kafka Connect writes to store config on startup |
| <code>connect-offsets</code> | Last processed offset tracking      | Updates on every message processed              |
| <code>connect-status</code>  | Task health and status              | Heartbeats sent continuously                    |

**Solution:** Deploy a separate local Kafka cluster within OpenShift exclusively for Kafka Connect state. This cluster:

- Has 1 replica (dev/test) or 3 replicas (production)
- Does NOT replicate business data
- Hosts only Kafka Connect internal topics
- Maintains independent lifecycle from data ingestion

Why Separate Connectors per Database?

Two JDBC Sink connectors (one for Db2, one for MSSQL) provide:

| Benefit               | Details                                                                         |
|-----------------------|---------------------------------------------------------------------------------|
| Failure isolation     | Db2 failure doesn't stall MSSQL ingestion                                       |
| Independent tuning    | Db2 can use <code>batch.size: 1</code> , MSSQL uses <code>batch.size: 10</code> |
| Separate error queues | Db2 DLQ and MSSQL DLQ tracked independently                                     |
| Resource isolation    | Each connector has own task pool, JVM heap, DB connections                      |

System Components

1. Kafka Connect Cluster (Strimzi-Managed)

**Component:** `KafkaConnect` custom resource in OpenShift  
**Replicas:** 1 (dev/test) or 3 (production)  
**Image:** Custom Debezium with plugins and drivers baked in

Plugins Included

```
# Stage 1: Build the Maven project
FROM images.pkgrepo.bcbssc.com/openshift/base-images/ansible:latest AS build

WORKDIR /build
```

```

# Copy the entire project
COPY . .

# Build the project using the internal settings.xml
# Get the Decryption SMT , aws msk library , debezium jdbc sink plugin ,
jdbc drivers and license for mssql and db2 from repository
RUN wget https://pkgrepo.bcbssc.com/artifactory/bcbssc-IS-maven-pre-
approval/com/bcm/tda/kafka/handler/CustomIAMHandler/1.0.1/CustomIAMHandler-
1.0.1.jar && \
    wget https://pkgrepo.bcbssc.com/artifactory/bcbssc-IS-maven-pre-
approval/software/amazon/msk/aws-msk-iam-auth/2.3.2/aws-msk-iam-auth-
2.3.2.jar && \
    wget https://maven.repository.redhat.com/ga/io/debezium/debezium-
connector-jdbc/3.4.3.Final-redhat-00001/debezium-connector-jdbc-
3.4.3.Final-redhat-00001-plugin.zip && \
    wget https://repo1.maven.org/maven2/com/microsoft/sqlserver/mssql-
jdbc/13.4.0.jre11/mssql-jdbc-13.4.0.jre11.jar && \
    wget https://pkgrepo.bcbssc.com/artifactory/bcbssc-IS-maven-pre-
approval/com/btds/smt-decryption/1.0.1/smt-decryption-1.0.1.jar && \
    wget https://pkgrepo.bcbssc.com/artifactory/bcbssc-IS-maven-vendor-
libs/com/ibm/db2/db2jcc_license_cisuz/10.5.0/db2jcc_license_cisuz-
10.5.0.jar && \
    wget https://pkgrepo.bcbssc.com/artifactory/bcbssc-IS-maven-vendor-
libs/com/ibm/db2/db2jcc4/10.5.0/db2jcc4-10.5.0.jar

RUN mkdir -p /opt/kafka/plugins/debezium-jdbc

# Stage 2: Create the final Debezium image
FROM registry.redhat.io/amq-streams/kafka-42-rhel9:3.2.0

USER root

COPY --from=build /build/CustomIAMHandler-1.0.1.jar /opt/kafka/libs/
COPY --from=build /build/CustomIAMHandler-1.0.1.jar /opt/kafka/plugins/aws-
msk-iam/
COPY --from=build /build/aws-msk-iam-auth-2.3.2.jar /opt/kafka/libs/

## Copying the sink plugins

RUN mkdir -p /opt/kafka/plugins/debezium-jdbc

COPY --from=build \
    /build/debezium-connector-jdbc-3.4.3.Final-redhat-00001-plugin.zip \
    /tmp/debezium-jdbc.zip

RUN unzip /tmp/debezium-jdbc.zip -d /opt/kafka/plugins/debezium-jdbc

COPY --from=build \
    /build/mssql-jdbc-13.4.0.jre11.jar \
    /opt/kafka/plugins/debezium-jdbc/
COPY --from=build /build/db2jcc_license_cisuz-10.5.0.jar
/opt/kafka/plugins/debezium-jdbc/debezium-connector-jdbc

```

```
COPY --from=build /build/db2jcc4-10.5.0.jar /opt/kafka/plugins/debezium-jdbc/debezium-connector-jdbc

COPY --from=build /build/smt-decryption-1.0.1.jar /opt/kafka/plugins/debezium-jdbc/

# Ensure proper permissions
#RUN chown -R strimzi:strimzi /opt/kafka/libs/

USER strimzi
```

## JVM Configuration

```
jvmOptions:
  heap: "2Gi" # Min for dev/test; 4Gi for production
  gc: "-XX:+UseG1GC -XX:MaxGCPauseMillis=20"
  system_properties:
    - http.proxyHost: ""
    - https.proxyHost: ""
    - http.nonProxyHosts: "*"
```

## 2. Custom SMTs (Single Message Transforms)

### A. Decryption SMT (`com.btds.Decryption`)

**Purpose:** Decrypt encrypted fields using AES-256-GCM

**Key Resolution:** Per-topic key mapping stored in mounted Kubernetes Secret

#### Configuration:

```
transforms.decrypt.type: com.btds.Decryption
transforms.decrypt.key.dir: /mnt/opt/kafka/decryption-keys
transforms.decrypt.topic.key.map:
"btds_plt_lh027_cr_*:lh027,btds_plt_lh030_cr_*:lh030,btds_plt_lh044_cr_*:lh044,btds_plt_lh045_cr_*:lh045"
```

#### Implementation Details:

- Reads Base64-encoded AES keys from directory
- Uses `JsonConverter.toConnectData()` to preserve Debezium logical types
- Moved `JsonConverter` instantiation to `configure()` to reduce per-record GC pressure
- Supports wildcard topic matching for license group scale-out

#### Key Storage (Kubernetes Secret):

```

apiVersion: v1
kind: Secret
metadata:
  name: decryption-keys
  namespace: openshift-operators
type: Opaque
data:
  lh027: <base64-encoded-key>
  lh030: <base64-encoded-key>
  lh044: <base64-encoded-key>
  lh045: <base64-encoded-key>

```

## B. ReplaceField\$Value SMT (Built-in)

**Purpose:** Rename Debezium CDC audit columns to Db2-compatible identifiers

**Why:** DB2 for z/OS rejects un delimited identifiers starting with \_\_

### Configuration:

```

transforms.renamefields.type:
org.apache.kafka.connect.transforms.ReplaceField$Value
transforms.renamefields.renames:
__deleted:AUDIT_DELETED,__table:AUDIT_TABLE,__schema:AUDIT_SCHEMA,__db:AUDI
T_DB,__op:AUDIT_OP,__ts_ms:AUDIT_TS_MS,__txId:AUDIT_TX_ID,__lsn:AUDIT_LSN

```

### Field Mappings:

| Source Field | Target Field  | Type   | Meaning                                          |
|--------------|---------------|--------|--------------------------------------------------|
| __deleted    | AUDIT_DELETED | string | Record deleted flag ("true"/"false")             |
| __table      | AUDIT_TABLE   | string | Source table name (e.g., "ipp_bill_dtl_history") |
| __schema     | AUDIT_SCHEMA  | string | Source schema (e.g., "its_plt_cr")               |
| __db         | AUDIT_DB      | string | Source database (e.g., "its_db")                 |
| __op         | AUDIT_OP      | string | Operation (c=create, u=update, d=delete, r=read) |
| __ts_ms      | AUDIT_TS_MS   | int64  | Event timestamp (milliseconds since epoch)       |
| __txId       | AUDIT_TX_ID   | int64  | Transaction ID                                   |
| __lsn        | AUDIT_LSN     | int64  | Log Sequence Number (replication position)       |

## 3. JDBC Sink Connector

**Component:** Debezium `JdbcSinkConnector`

**Role:** Consumes transformed Kafka records and writes to target databases

## Connection Configuration

```
connection.url: "jdbc:db2://zos-host:1234/ITS_DB" # Db2
# OR
connection.url: "jdbc:sqlserver://mssql-host:1433;database=staging" #
MSSQL

connection.username: ${strimzisecrets:namespace/secret:key}
connection.password: ${strimzisecrets:namespace/secret:key}
```

## Write Mode Configuration

```
insert.mode: insert # Generate Insert
table.name.format: ITS_DEV_CR.${topic} # Target schema.table
collection.name.format: ${topic} # Debezium 3.x (replaces
table.name.format)
schema.evolution: none # DBAs own DDL, no auto-
schema-changes
```

## 4. Database Staging Tables

**Constraints:** NONE (critical design decision)

**Replication Factor:** N/A (no primary keys to replicate)

**Purpose:** Audit trail and data warehouse source, not transactional store

### Why No Constraints?

1. **CDC doesn't provide key information:** Debezium JDBC sink lacks primary key metadata from source
2. **Out-of-order operations:** UPDATE can arrive before CREATE, violating FK constraints
3. **Staging semantics:** Tables meant to capture all changes for audit, not enforce consistency
4. **Simplified recovery:** DBAs can truncate and resync without constraint violations

### Table Structure Example (Db2)

```
CREATE TABLE ITS_DEV_CR.IPP_BILL_DTL_HISTORY (
  bill_dtl_serial_num    VARCHAR(30) NOT NULL,
  bus_owner_id           VARCHAR(10),
  part_key_num           SMALLINT,
  -- ... business columns ...
  cnstnt_memb_id         VARCHAR(30),
  actl_sub_apfx_id       VARCHAR(10),

  -- Audit columns (renamed from __ prefix)
  AUDIT_DELETED          VARCHAR(5),
  AUDIT_TABLE            VARCHAR(64),
```

```

    AUDIT_SCHEMA          VARCHAR(64),
    AUDIT_DB              VARCHAR(64),
    AUDIT_OP              VARCHAR(1),
    AUDIT_TS_MS           BIGINT,
    AUDIT_TX_ID           BIGINT,
    AUDIT_LSN             BIGINT
)
TABLESPACE ITS_DEV_TS_GROWTH
PARTITION BY GROWTH;

```

**Note:** No PRIMARY KEY, UNIQUE, or FOREIGN KEY constraints.

## 5. AWS MSK Consumer Configuration

**Source:** AWS Managed Streaming for Kafka (read-only)

**Authentication:** AWS IAM with signature v4

### IAM Configuration

```

consumer.override.bootstrap.servers: "pkc-
xxx.region.provider.confluent.cloud:9092"
consumer.override.security.protocol: "SASL_SSL"
consumer.override.sasl.mechanism: "AWS_MSK_IAM"
consumer.override.sasl.jaas.config:
"software.amazon.msk.auth.iam.IAMLoginModule required;"
consumer.override.sasl.client.callback.handler.class:
"com.bcm.tda.kafka.handler.CustomIAMClientCallbackHandler"
consumer.override.ssl.endpoint.identification.algorithm: ""
consumer.override.ssl.enabled.protocols: "TLSv1.2,TLSv1.3"

```

### Custom IAM Callback Handler

**Component:** CustomIAMClientCallbackHandler

#### Improvements Made:

- Token caching (reuse tokens for 55 minutes, refresh at 50 minutes)
- Connection-level SSL socket factory (avoid per-thread creation overhead)
- Interrupt-aware retry logic with exponential backoff

#### Configuration:

```

public class CustomIAMClientCallbackHandler implements
AuthenticateCallbackHandler {
    private static final Holder<IAIAuthenticator> authenticator = new
LazyHolder<>(...);
    private static final AtomicBoolean configured = new AtomicBoolean(false);
    private static final ReentrantLock thunder_herd_lock = new
ReentrantLock();

```



```
// Token reuse: 55 min validity, refresh at 50 min
private static final long TOKEN_REUSE_THRESHOLD_MS = 50 * 60 * 1000L;

// Prevents thundering herd on token refresh
// Retry with backoff: 100ms, 200ms, 400ms, 800ms (max)
}
```

## Deployment Architecture

### Kubernetes Resource Hierarchy

#### OpenShift Cluster

- └─ Namespace: tenant-comm-debezium(db2)
  - └─ KafkaConnect (1 or 2 clusters)
    - └─ bt-ds-db2-connect (1 pods)
  - └─ KafkaConnector (1 or 2 clusters)
    - └─ bt-ds-msdb2sql-connector (1 pods)
  - └─ Secrets
    - └─ db2-credentials (username/password)
    - └─ debezium-msk-iam-cert
    - └─ decryption-keys
  - └─ RoleBinding
    - └─ connect-secret-reader
- └─ Namespace: tenant-comm-debezium-mssql (mssql\_)
  - └─ KafkaConnect (local broker)
    - └─ bt-ds-mssql-connect (1 pods)
  - └─ KafkaConnector (1 or 2 clusters)
    - └─ bt-ds-mssql-connector (1 pods)
  - └─ Secrets
    - └─ mssql-database-secret (username/password)
    - └─ debezium-msk-iam-cert
    - └─ decryption-keys
  - └─ RoleBinding: connect-secret-reader
  - └─ RoleBinding
    - └─ connect-secret-reader
- └─ Namespace: tenant-comm-kafka (broker)
  - └─ Kafka (local broker)
    - └─ KafkaTopic: db2-connect-configs
    - └─ KafkaTopic: db2-connect-offsets
    - └─ KafkaTopic: db2-connect-status
    - └─ KafkaTopic: mssql-connect-status
    - └─ KafkaTopic: mssql-connect-status
    - └─ KafkaTopic: mssql-connect-status
    - └─ KafkaTopic: db2-sink-dlq
    - └─ KafkaTopic: mssql-sink-dlq
- └─ Namespace: tentant-comm-debezium-build (build namesapce)
  - └─ build namespace to run helm ( can be replaced with tekton pipeline)

## Helm Chart Structure

```
kafka

kafkaconnect/
├── values.yaml                # Global configuration
├── Chart.yaml                # Chart metadata
├── templates/
│   ├── 10-kafkanaodepoolcontroller.yaml    # Kafka Node pool Controller
Object ( pre-install )
│   ├── 10-kafkanaodepoolbroker.yaml        # Kafka Node pool Broker Object
( pre-install )
│   ├── 10-kafka.yaml                    # Kafka with Kraft ( pre-install
)
│   ├── 10-topics.yaml                    # Kafka topics needed for the
connect and connector
kafkaconnect/
├── values.yaml                # Global configuration
├── Chart.yaml                # Chart metadata
├── templates/
│   ├── 10-mssqlcredentials.yaml    # MS SQL Database Secret (pre-
install)
│   ├── 10-decryption-keys.yaml    # License Group Decryption Keys
Secret (pre-install)
│   ├── 10-db2credentials.yaml      # DB2 Database Secret(pre-install)
│   ├── 10-certificatesecret.yaml  # AWS HTTP call certificate
RoleBinding (Pre-install)
│   ├── 20-kafkaconnect.yaml        # Kafka Connect Object to be
installed in both namespaces connector (post-install)
│   ├── 25-role.yaml                # Role for accessing secrets in
connector (pre-install )
│   ├── 25-rolebinding.yaml         # Rolebinding to the service account
for accessing secrets in connector (pre-install )
│   ├── 30-db2connector.yaml        # Db2 Kafka Debezium connector
(post-install)
│   ├── 30-mssqlconnector.yaml      # MSSQL Kafka Debezium connector
(post-install)
│   └── README.md
└── README.md
```

## Configuration Reference

### Kafka Connect Cluster (10-kafkaconnect.yaml)

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaConnect
metadata:
  name: bt-ds-db2-connect
  namespace: openshift-operators
annotations:
```

```
    strimzi.io/use-connector-resources: "true"
spec:
  replicas: 3 # 1 (dev), 3+ (prod)
  image: quay.io/company/debezium:3.0-custom
  bootstrapServers: "local-kafka:9092"

  livenessProbe:
    httpGet:
      path: /health
      port: rest-api
    initialDelaySeconds: 300
    periodSeconds: 120
    failureThreshold: 3

  logging:
    type: inline
    loggers:
      rootLogger.level: INFO
      logger.kafka.name: org.apache.kafka
      logger.kafka.level: INFO
      logger.debezium.name: io.debezium.connector.jdbc
      logger.debezium.level: DEBUG # For production troubleshooting

  resources:
    requests:
      memory: "1Gi" # Dev/test minimum
      cpu: "500m"
    limits:
      memory: "4Gi" # Production
      cpu: "2"

  jvmOptions:
    javaSystemProperties:
      - name: http.proxyHost
        value: ""
      - name: https.proxyHost
        value: ""
      - name: http.nonProxyHosts
        value: "*"

  template:
    pod:
      volumes:
        - name: msk-iam-cert
          secret:
            secretName: msk-iam-cert
        - name: db-credentials
          secret:
            secretName: db-credentials
        - name: decryption-keys
          secret:
            secretName: decryption-keys

  connectContainer:
```

```

    volumeMounts:
      - name: db-credentials
        mountPath: /mnt/db/
      - name: decryption-keys
        mountPath: /mnt/opt/kafka/decryption-keys
        readOnly: true

  config:
    connector.client.config.override.policy: All
    config.storage.replication.factor: 1
    offset.storage.replication.factor: 1
    status.storage.replication.factor: 1
    offset.storage.partitions: 25
    status.storage.partitions: 5

```

## DB2 Connector (30-db2connector.yaml)

```

apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaConnector
metadata:
  name: bt-ds-db2-connector
  namespace: data-pipeline
spec:
  class: io.debezium.connector.jdbc.JdbcSinkConnector

  config:
    # ===== AWS MSK Consumer =====
    # MSK broker addresses (IAM-authenticated) , This is not local broker
    address
    consumer.override.bootstrap.servers: "msk.amazonaws.com:9092"
    # Consumer group ID for offset tracking
    consumer.override.group.id: "bt-ds-db2-connector-0"
    # SASL_SSL = TLS encryption + AWS IAM authentication

    consumer.override.security.protocol: "SASL_SSL"
    consumer.override.sasl.mechanism: "AWS_MSK_IAM"
    consumer.override.sasl.jaas.config:
"software.amazon.msk.auth.iam.IAMLoginModule required;"
    consumer.override.sasl.client.callback.handler.class:
"com.bcm.tda.kafka.handler.CustomIAMClientCallbackHandler"
    consumer.override.ssl.endpoint.identification.algorithm: ""
    consumer.override.ssl.enabled.protocols: "TLSv1.2,TLSv1.3"

    # This above section was put together based on the on-boarding document
    provided by the BCA team

    # ===== Database Connection =====

    connection.url: "<db2Connecton url from values file>"
    #Strimzi config provides multiple ways to read secrets from environment
    , openshift secrets , files or directories

```

```

#it is defined by the configuration whose default value is
#config.providers=strimzienv,strimzifile,strimzidir,strimzisecrets
# we use strimzisecrets: and the syntax is
configprefix:namespace/secretname/key
  connection.username: ${strimzisecrets:db2connectornamespace/db-
credentials:db2-username}
  connection.password: ${strimzisecrets:db2connectornamespace/db-
credentials:db2-password}

# ===== Topic Subscription =====
#The topics to consume cdc event from upstream
topics: "<list of topics>"

# ===== Write Mode =====
# We use insert mode to process data in to the database , this is
driven from the statement in database design
# which states that the data ingestion database must be treated as
staging and audit columns must be captured
  insert.mode: insert
# The name of the table mapped from the topic name and any SMT's and
schema
  table.name.format: ITS_DEV_CR.${topic}

# ===== Schema Management =====
# As guided by the source team in DB design
schema.evolution: none

# ===== Transform Chain =====
# ORDER CRITICAL: topic cleanup → decrypt → rename fields
# These are SMT's configured and written to transform the CDC record
before it gets inserted to the database
# renametopic : Inbuilt in Kafka Connector , renames a topic using
regular expression and replacement and derives the table name
# decrypt : Derived from the sample code provided by the Source team to
decrypt the incoming payload using a decryption key
#           The code was modified to read from a map to derive the
decryption key based on the license group the topic belong to
#           defined with the field transforms.decrypt.topic.key.map
# renamefields: Inbuilt SMT , renames the feilds from old value to a
new value provided by the config transforms.renamefields.renames
#           This is needed for mapping the audit columns with
__prefix to AUDIT__ to enable the field name mapping and preferred over
#           preserve quoted config as there was an issue in
converting the table name.
  transforms: renametopic,decrypt,renamefields

# configuration or the rename SMT
transforms.renametopic.type:
org.apache.kafka.connect.transforms.RegexRouter
transforms.renametopic.regex: "^btds_plt_lh[0-9]+_cr_(.)_cout$"
transforms.renametopic.replacement: "$1"

# configuration for the decryprion SMT

```

```

transforms.decrypt.type: com.btds.Decryption
transforms.decrypt.key.dir: "/mnt/opt/kafka/decryption-keys"
transforms.decrypt.topic.key.map:
"btds_plt_lh027_cr_*:lh027,btds_plt_lh030_cr_*:lh030,btds_plt_lh044_cr_*:lh
044,btds_plt_lh045_cr_*:lh045"

# Configuration for the renamefields SMT
transforms.renamefields.type:
org.apache.kafka.connect.transforms.ReplaceField$Value
transforms.renamefields.renames:
"__deleted:AUDIT_DELETED,__table:AUDIT_TABLE,__schema:AUDIT_SCHEMA,__db:AUD
IT_DB,__op:AUDIT_OP,__ts_ms:AUDIT_TS_MS,__txId:AUDIT_TX_ID,__lsn:AUDIT_LSN"

# ===== Converters =====
key.converter: org.apache.kafka.connect.json.JsonConverter
key.converter.schemas.enable: true
value.converter: org.apache.kafka.connect.storage.StringConverter
value.converter.schemas.enable: false

# ===== Batch Processing =====
# use this when ordering of the messages becomes important and relavant
, as the current DB is staging we are
# using batch mode , also caution should be used when using this
paramter , performance will be slow
#batch.size: 1
linger.ms: 1000
max.retries: 3
retry.backoff.ms: 100

# ===== Error Handling =====
errors.tolerance: all
errors.retry.timeout: 60000
errors.retry.delay.max.ms: 5000
errors.log.enable: true
errors.log.include.messages: true
errors.deadletterqueue.topic.name: "btds-db2-dlq"
errors.deadletterqueue.context.headers.enable: true

tasksMax: 1 # Single task per connector

```

## Data Flow & Transformations

### Record Lifecycle

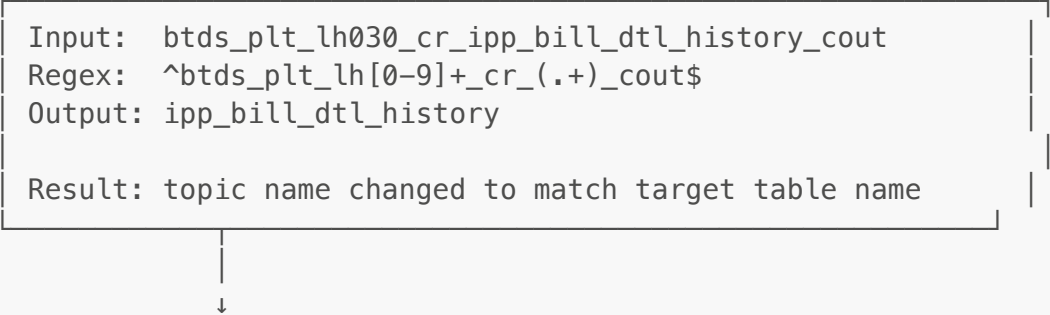
↓

Stage 1: AWS MSK Topic (Encrypted)

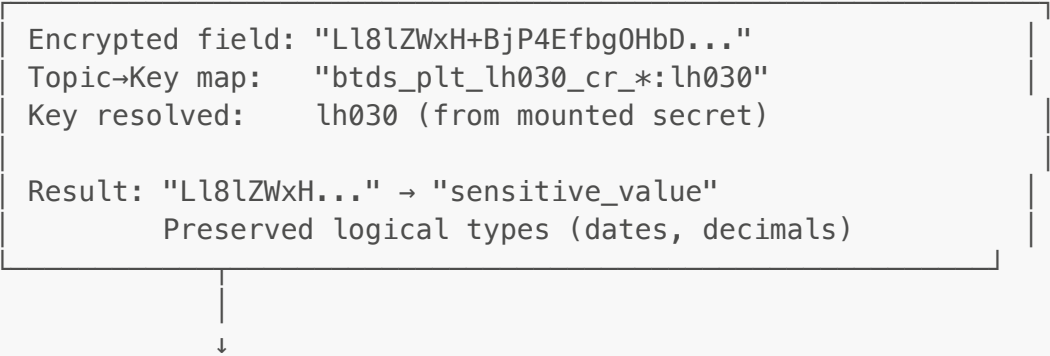
|                                                                    |
|--------------------------------------------------------------------|
| Topic: btds_plt_lh030_cr_ipp_bill_dtl_history_cout<br>Partition: 2 |
|--------------------------------------------------------------------|



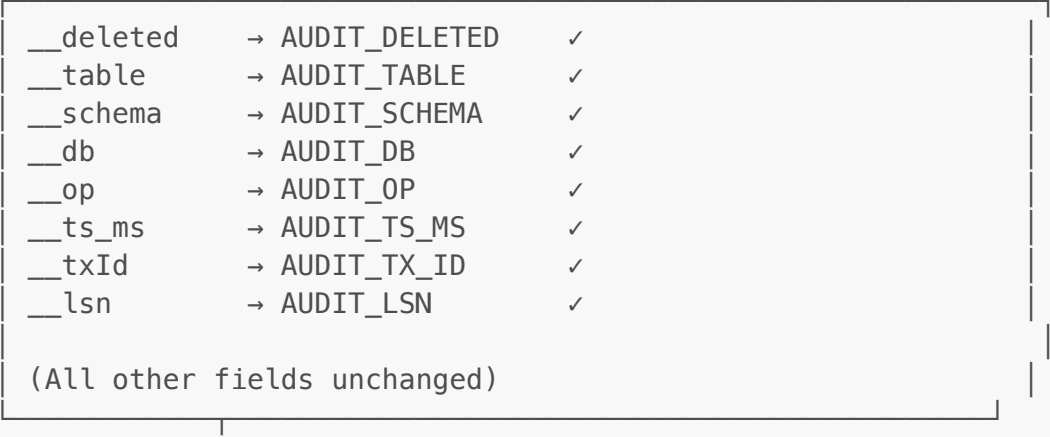
Stage 3: Transform 1 – RegexRouter (Topic Cleanup)



Stage 4: Transform 2 – Decryption (AES-256-GCM)



Stage 5: Transform 3 – ReplaceField\$Value (Rename Audit Columns)



↓

### Stage 6: JDBC Sink Connector (Write to Db2)

Generated SQL:

```
INSERT INTO ITS_DEV_CR.ipp_bill_dtl_history T  
VALUES (...)
```

Result: Row inserted with renamed audit columns

↓

### Stage 7: Target Database

Db2 Table: ITS\_DEV\_CR.IPP\_BILL\_DTL\_HISTORY

AUDIT\_DELETED: false

AUDIT\_TABLE: ipp\_bill\_dtl\_history\_56

AUDIT\_OP: c

AUDIT\_TS\_MS: 1780967739146

AUDIT\_TX\_ID: 119034065

AUDIT\_LSN: 100530672216

## Schema Reconciliation

**Scenario:** Kafka record schema vs. database table structure

1. **Debezium generates schema** from source table DDL (includes all 8 audit columns)
2. **ReplaceField\$Value renames** `__op` → `AUDIT_OP` (in record only)
3. **JDBC sink checks table schema** to determine if INSERT or UPDATE needed
4. **Hibernate ORM generates SQL** using transformed field names
5. **Db2 accepts** `AUDIT_*` columns (valid identifiers)

**Critical Point:** With `schema.evolution: none`, the target table DDL must pre-exist with `AUDIT_*` column names. The connector does NOT auto-create tables or ALTER TABLE for renamed columns.

---

## Known Issues & Resolutions

Issue #1: Db2 ERRORCODE=-4229 (Invalid Identifier)

**Symptoms:**

```
[jcc][t4][102][10040][4.19.66] Batch failure. The batch was submitted, but  
at least
```



```
one exception occurred on an individual member of the batch.  
ERRORCODE=-4229, SQLSTATE=null
```

**Root Cause:**

Debezium CDC audit columns (`__deleted`, `__op`, etc.) use underscore prefix. Db2 for z/OS does not allow unlimited identifiers starting with underscore.

**Resolution:**

Apply `ReplaceField$Value` SMT to rename all 8 audit columns from `__` prefix to `AUDIT_` prefix.

**Implementation:**

```
transforms.renamefields.type:  
org.apache.kafka.connect.transforms.ReplaceField$Value  
transforms.renamefields.renames:  
__deleted:AUDIT_DELETED,__table:AUDIT_TABLE,__schema:AUDIT_SCHEMA,__db:AUDI  
T_DB,__op:AUDIT_OP,__ts_ms:AUDIT_TS_MS,__txId:AUDIT_TX_ID,__lsn:AUDIT_LSN
```

**Verification:**

```
# Check Kafka Connect logs  
oc logs -f pod/btds-db2-connect-0 | grep -i "ERRORCODE=-4229"  
  
# Query Db2 table for renamed columns  
db2 "SELECT COUNT(*), AUDIT_OP, AUDIT_TABLE FROM  
ITS_DEV_CR.IPP_BILL_DTL_HISTORY GROUP BY AUDIT_OP, AUDIT_TABLE"
```

**Issue #2: Batch Ordering & Duplicate Key Violations****Symptoms:**

```
Duplicate entry 'bill_dtl_serial_num' for key 'PRIMARY'  
OR  
Cannot insert or update row in table because a duplicate key was found
```

**Root Cause:**

CDC operations on the same record may arrive out-of-order. Example:

- Offset 100: CREATE for `bill_serial_num='ABC123'`
- Offset 101: UPDATE for `bill_serial_num='ABC123'`

If two are batched together, the second update operation fails with duplicate key error as the operation is considered a new insert

**Resolution:**

remove the constraints on the database , this was identified as requirement after the issue.

**Issue #3: AWS MSK Read-Only Constraint****Symptoms:**

```
Authorization failed: User is not authorized to perform action
OR
write-only access denied to topic 'connect-offsets'
```

**Root Cause:**

AWS MSK broker policy restricts all writes to internal topics. Kafka Connect requires write access to:

- `connect-configs`
- `connect-offsets`
- `connect-status`

**Resolution:**

Deploy separate local Kafka cluster within OpenShift exclusively for Connect state.

**Architecture:**

```
# Local Kafka (in-cluster)
bootstrap.servers: "kafka-local:9092"
config.storage.topic: "connect-configs"
offset.storage.topic: "connect-offsets"
status.storage.topic: "connect-status"

# AWS MSK (external, read-only)
consumer.override.bootstrap.servers: "msk.amazonaws.com:9092"
consumer.override.security.protocol: "SASL_SSL"
consumer.override.sasl.mechanism: "AWS_MSK_IAM"
```

**Issue #4: Db2 Tablespace Partition Issues****Symptoms:**

```
SQLCODE=-254 'Cannot use column...of type...in tablespace...'
OR
'Tablespace does not support column types'
```

**Root Cause:**

Db2 for z/OS uses `PARTITION BY RANGE` for some tablespaces, which has strict type requirements.

**Resolution:**

Convert tablespace from **PARTITION BY RANGE** to **PARTITION BY GROWTH**.

**SQL:**

```
-- Check current partitioning
SELECT TBSPACE, PAGESIZE, PARTITIONTYPE FROM SYSCAT.TABLESPACES WHERE
TBSPACE='ITS_DEV_TS';

-- Create new tablespace with GROWTH partitioning
CREATE TABLESPACE ITS_DEV_TS_GROWTH PAGESIZE 4K PARTITION BY GROWTH;

-- Reorganize tables to new tablespace
ALTER TABLE ITS_DEV_CR.IPP_BILL_DTL_HISTORY MOVE TO TABLESPACE
ITS_DEV_TS_GROWTH;
```

---

## Operational Procedures

### Deployment

#### Prerequisites

- Red Hat OpenShift 4.8+ with cluster-admin access
- Strimzi Operator installed in **openshift-operators** namespace
- Helm 3.x
- Network access to AWS MSK and target databases

#### Step 1: Deploy via Helm

```
# Install/upgrade release

To install Kafka local broker
helm upgrade --install kafka . \
  --namespace openshift-operators \
  --values ./values.yaml \
  --wait \
  --timeout 10m
```

#### Step 2: Verify Deployment

```
oc get knp -n kafkanamespace oc get kp -n kafkanamespace oc get kafkatopic -n kafkanamespace
```

verify the broker is up and running and also check for the entity operator pod , this ensures that the operator is watching the namespace

To install Kafka connector and connect

```
helm upgrade --install kafkaconnector. \
  --namespace openshift-operators \
  --values ./values.yaml \
  --wait \
  --timeout 10m
```

### Step 3: Verify Deployment

```
# Check Kafka Connect cluster
oc get kafkaconnect -n db2namespace
oc get kafkaconnect -n mssqlnamespace

# Check Kafka topics created

# Check connector status
oc describe kc btds-db2-connector -n openshift-operators
oc logs -f pod/btds-db2-connect-0 -n openshift-operators | head -50
```

**END OF DOCUMENT**